REZERVACE KONZULTACÍ



Aplikace slouží jako **rezervační systém konzultací mezi studenty a pedagogy. Pedagogové** mohou vytvářet a spravovat dostupné termíny, zatímco **studenti** si vybírají vhodný čas a odesílají žádost o konzultaci

TÝM :

KB

Kseniia Blokhina

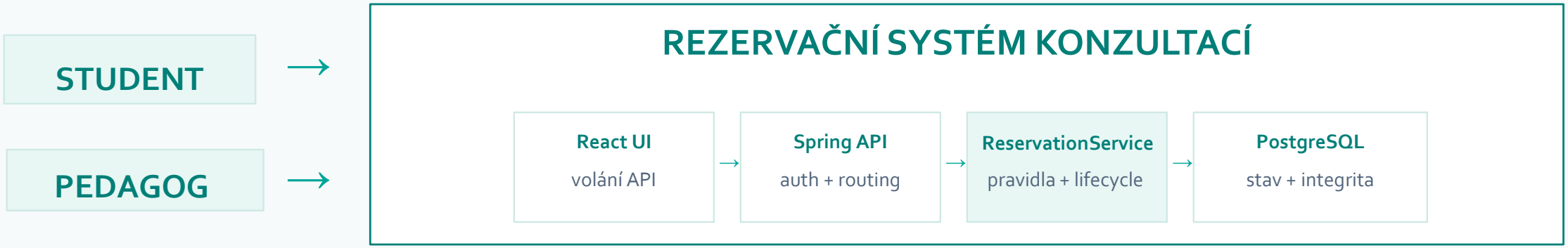
FT

Fatima Tangatarova

Rozsah a přijaté chování

Vybraná operace C02 OP-01 / REQ-01: student vytvoří rezervaci.

HRANICE SYSTÉMU · ROLE A ODPOVĚDNOSTI

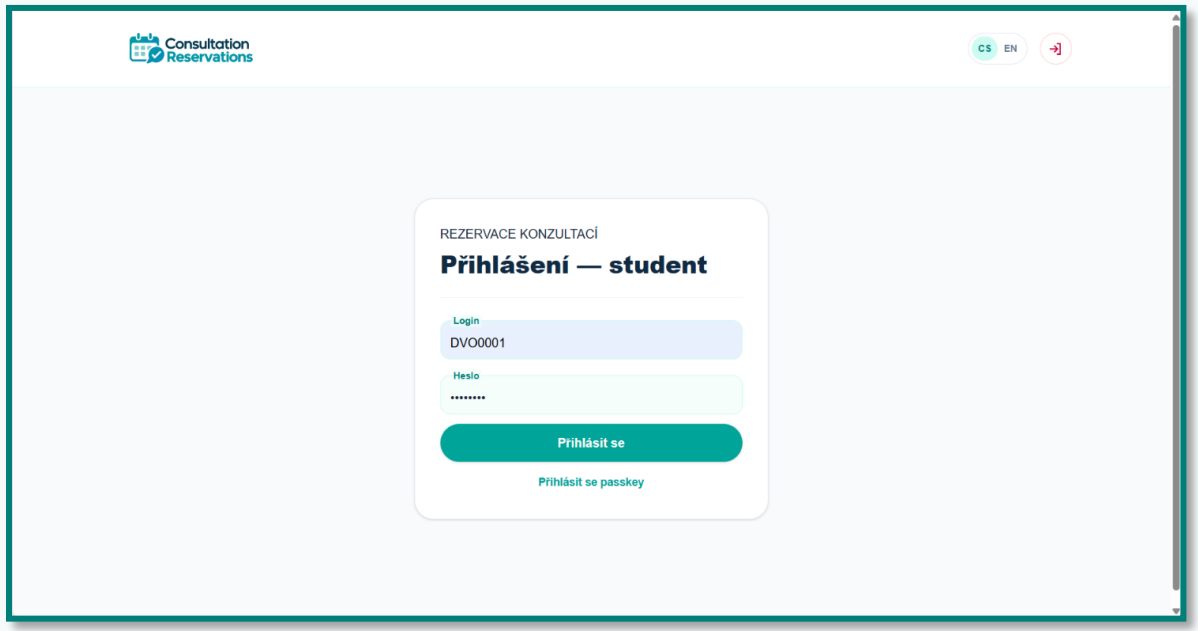


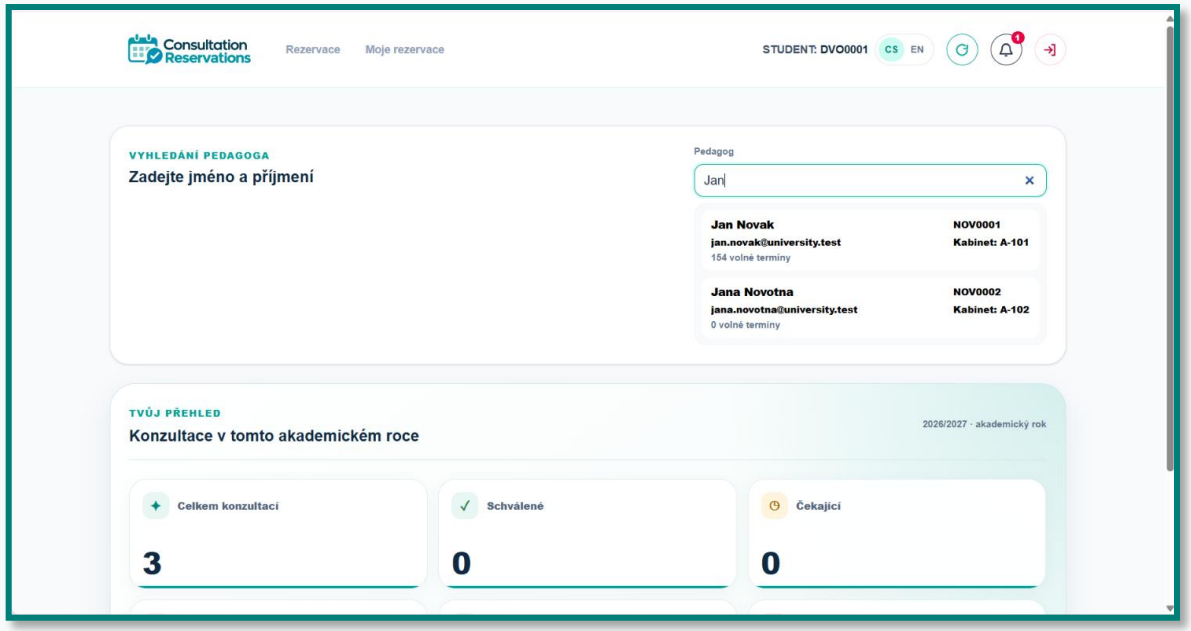
Student může:
zobrazit dostupné konzultační termíny,
vytvořit rezervaci,
zobrazit své rezervace,
zrušit vlastní aktivní rezervaci.

Pedagog může:
vytvářet a spravovat vlastní konzultační termíny,
zobrazit rezervace ke svým termínům,
schválit rezervaci svého termínu,
zrušit rezervaci nebo vlastní termín podle pravidel systému.

Rozsah a přijaté chování

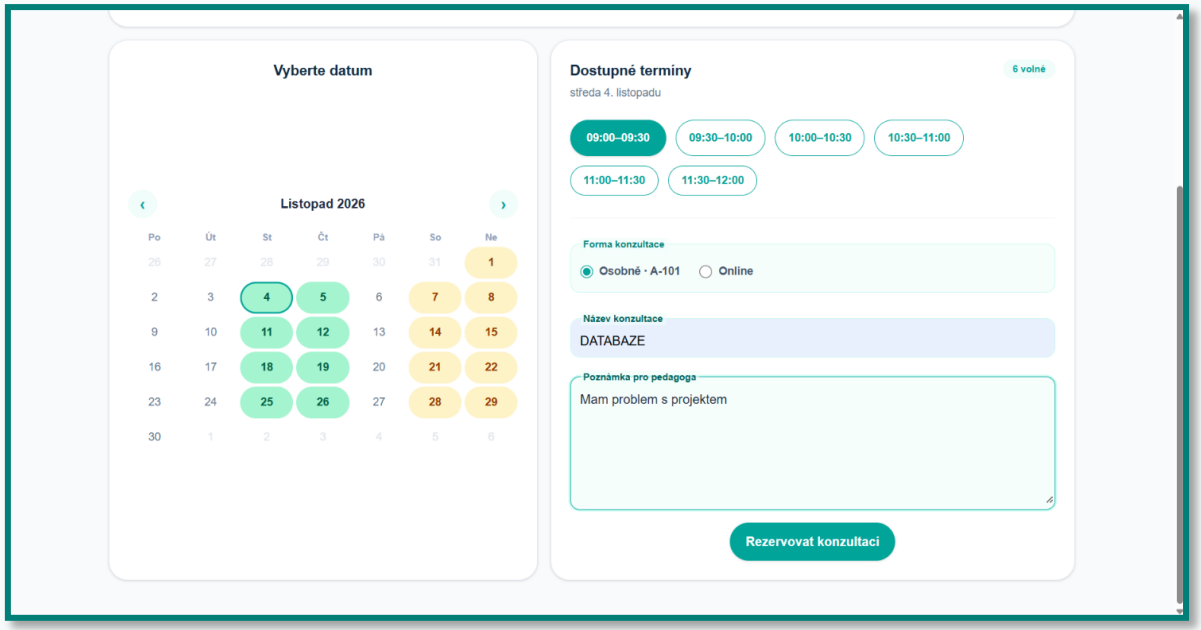
Vybraná operace C02 OP-01 / REQ-01: student vytvoří rezervaci.

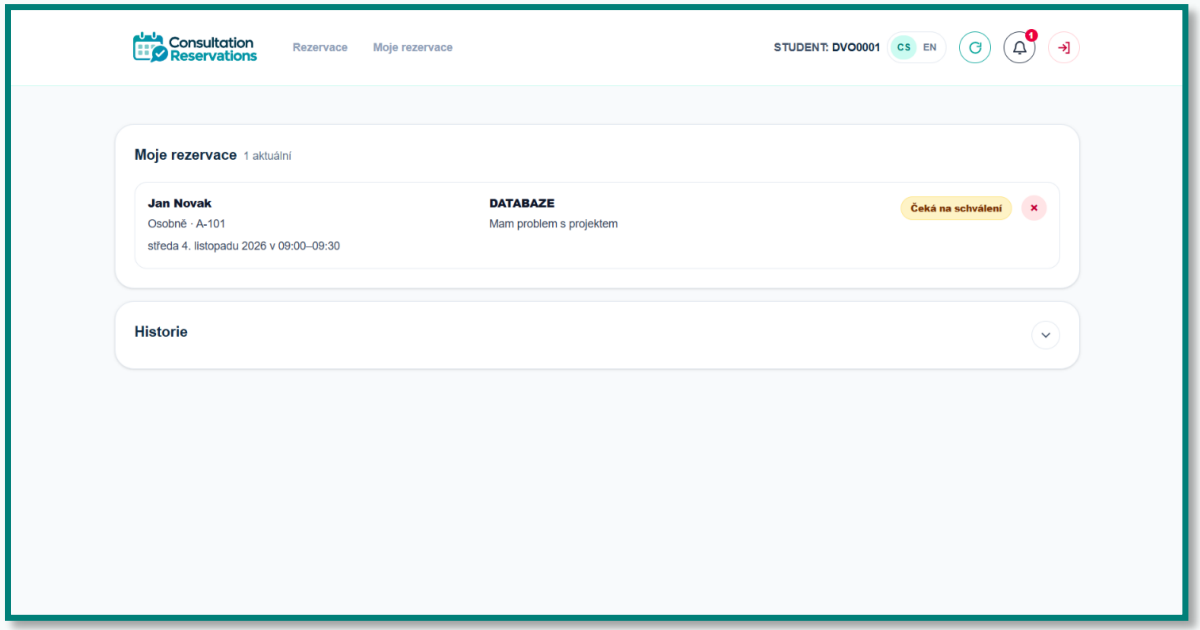




Rozsah a přijaté chování

Vybraná operace C02 OP-01 / REQ-01: student vytvoří rezervaci.



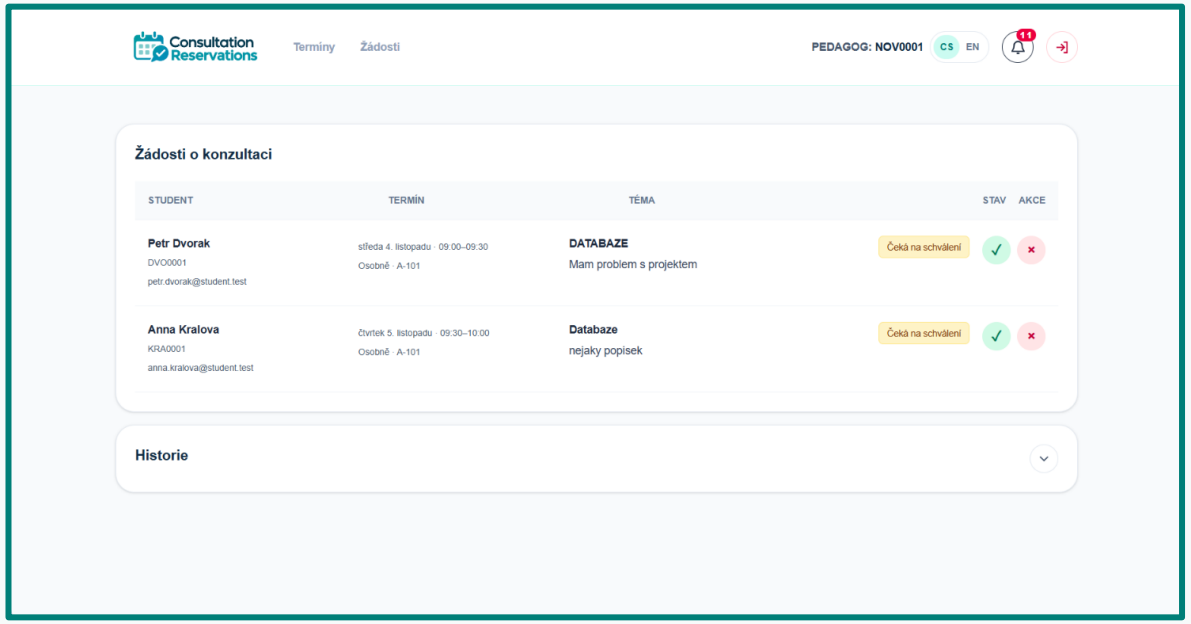


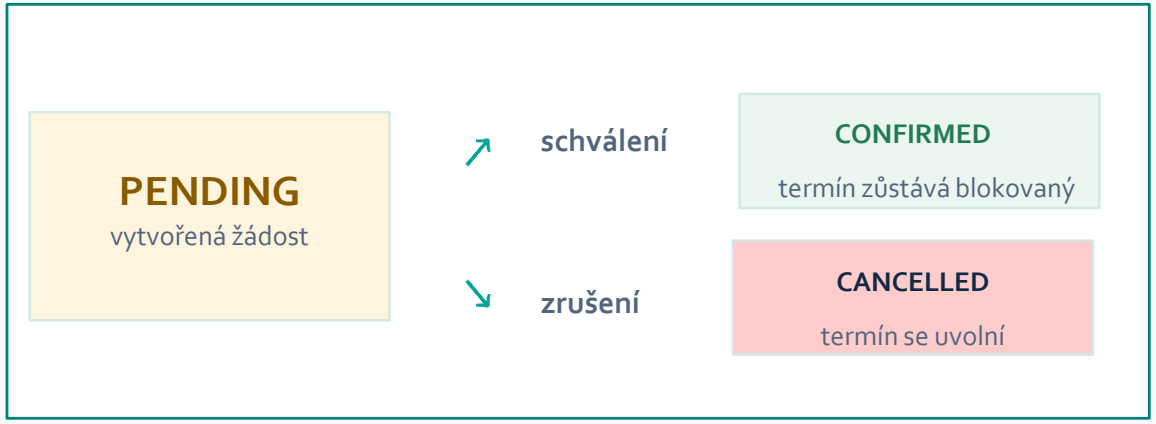
Rozsah a přijaté chování

Vybraná operace C02 OP-05 / REQ-05: approve reservation.

Před C02 se rezervace potvrzovala automaticky. C02 zavedlo samostatné rozhodnutí pedagoga. Stav PENDING stále blokuje termín, aby jej nemohl současně rezervovat jiný student.

Business invariant
Jeden termín může mít nejvýše jednu aktivní rezervaci.

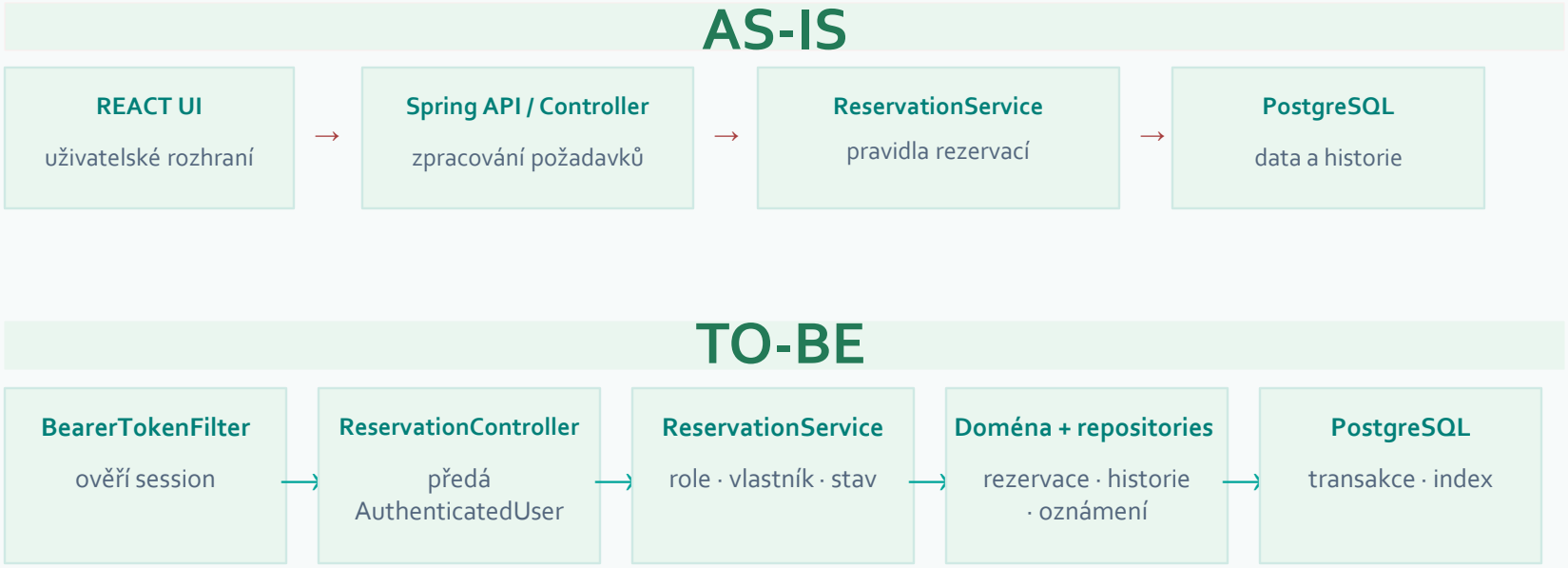




Architektura · komponenty a ADR

Driver: jeden termín může mít nejvýše jednu aktivní rezervaci

STATICKÝ POHLED · KOMPONENTY A VLASTNÍCI | AS-IS → TO-BE



Architektura · komponenty a ADR

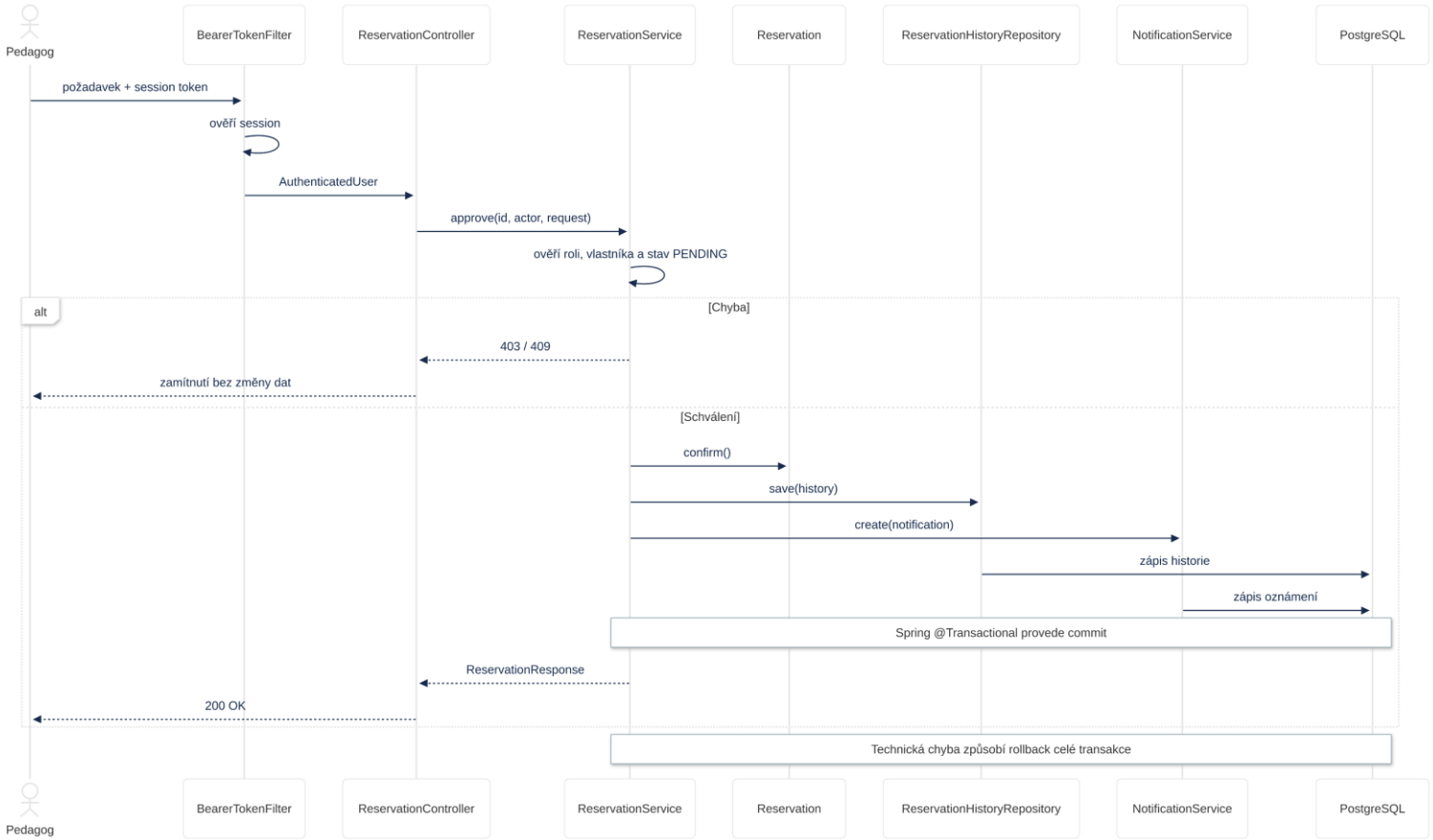
Driver: jeden termín může mít nejvýše jednu aktivní rezervaci

ALTERNATIVA A · transakční aplikační služba

ReservationService: ověří aktéra a pravidla, změní stav rezervace, zapíše historii a oznámení, vše provede v jedné transakci.

ALTERNATIVA B · databázově vlastněný workflow

Databázová funkce/procedura:
- rozhodne o změně stavu,
- zapíše rezervaci, historii a oznámení,
- backend pouze předá parametry a vrátí výsledek.



Detailní návrh · kontrakt a algoritmus

C05 jednotka: ReservationService.approve(id, aktér, request) · kontroly před zápisem.

PŘEDPOKLADY → PODMÍNKY PO ÚSPĚCHU

Před: ověřený PEDAGOG; ID a požadavek existují; rezervace existuje; aktér vlastní slot; status=PENDING. Po commitu: CONFIRMED + confirmedAt + historie + oznámení studentovi.

ROZHODOVACÍ TOK · PRVNÍ NESPLNĚNÁ PODMÍNKA VOLÁNÍ UKONČÍ

1 · IDENTITA	2 · ROLE	3 · VSTUP
aktér existuje?	PEDAGOG?	ID + požadavek?
ne → 401	jiná role → 403	null → 400

4 · NAČTENÍ	5 · VLASTNICTVÍ	6 · STAV
rezervace existuje?	slot.teacher == aktér?	PENDING?
ne → 404	cizí → 403	jiný stav → 409

ÚSPĚŠNÝ VĚTVÍ

Reservation.confirm() nastaví CONFIRMED a confirmedAt; službě zapíše historii a oznámení pod @Transactional. Poznámka requestu je volitelná.

```
public ReservationResponse approve(
    UUID reservationId,
    AuthenticatedUser authenticatedActor,
    ReservationTransitionRequest request) {
  if (authenticatedActor == null)
    throw new ApiException(HttpStatus.UNAUTHORIZED, "Authentication is required.");
  if (authenticatedActor.activeRole() != UserRole.TEACHER)
    throw new ApiException(HttpStatus.FORBIDDEN, "Only a teacher can approve a reservation.");
  if (reservationId == null || request == null)
    throw new ApiException(HttpStatus.BAD_REQUEST, "Reservation ID and request are required.");
  Reservation reservation = get(reservationId);
  AppUser teacher = users.get(authenticatedActor.id());
  if (!reservation.getSlot().getTeacher().getId().equals(teacher.getId()))
    throw new ApiException(HttpStatus.FORBIDDEN, "Only the slot's teacher can approve it.");
  if (reservation.getStatus() != ReservationStatus.PENDING)
    throw new ApiException(HttpStatus.CONFLICT, "Only a pending reservation can be approved.");
  reservation.confirm();
  history.save(
      new ReservationHistory(
          reservation,
          ReservationAction.CONFIRMED,
          ReservationStatus.PENDING,
          ReservationStatus.CONFIRMED,
          teacher,
          request.note()));
  notifications.create(
      reservation.getStudent(),
      reservation,
      "RESERVATION_APPROVED",
      "Rezervace byla schválena",
      "Pedagog "
          + teacher.getFirstName()
          + " "
          + teacher.getLastName()
          + " schválil vaši žádost o konzultaci.");
  return ReservationResponse.from(reservation);
}
```

CHYBOVÉ VÝSLEDKY

401/403/400/404/409 bez úspěšného výsledku. RuntimeException se propaguje; databázový rollback ověřují PostgreSQL testy.

Detailní návrh · hranice a failure

Služba rozhoduje; doména mění stav; Spring + PostgreSQL dokazují commit a rollback.

ODPOVĚDNOSTI A TRANSAKČNÍ HRANICE

ReservationService
autorizace · pořadí kontrol
koordinace lifecycle

→

Reservation
confirm() mění
stav + čas

→

Repositories
rezervace + historie
perzistence

→

NotificationService
oznámí studentovi
stejná transakce

@Transactional · VŠECHNY ZÁPISY

1 Reservation.confirm() → 2 historie.save() → 3 notifications.create() → 4 commit

```
consultation_reservation_backend.ReservationApprovalPersistenceTests
```

```
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0,
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------
[INFO] Total time:  26.151 s
[INFO] Finished at: 2026-11-02T21:25:26+01:00
[INFO] ------------------------------------------------
```

Verifikace a důkazy

Každý oracle je svázaný s aktuální C07 baseline a s hranicí konkrétního testu.

C04 · RIZIKO KVALITY → KRITÉRIUM → MĚŘENÍ

Bezpečnost a integrita auditu: bez session → 401; chybná role / cizí vlastník → 403 bez změny stavu, historie a oznámení. 14 API testů používá MockMvc, skutečný BearerTokenFilter, controller a službě; persistence je mockovaná.

TEST / ÚROVEŇ	OČEKÁVÁNÍ / ORACLE	POZOROVANÝ VÝSLEDEK
POZITIVNÍ · PostgreSQL successfulApprovalCommitsStateHistoryAndNotification	CONFIRMED + confirmedAt; přesně +1 historie a +1 oznámení se správným aktérem/příjemcem.	**PROŠEL · opětovné načtení z DB potvrdilo všechny zápisy.**
NEGATIVNÍ · unit test foreignTeacherCannotApproveEvenWhenStateIsInvalid	Cizí pedagog schvaluje CONFIRMED → 403, nikoli 409; žádná změna času/stavu ani auditní zápis.	**PROŠEL · aserce ověřily prioritu 403 i absenci zápisu.**
HRANIČNÍ · PostgreSQL concurrentApprovalsCommitOnlyOneLifecycleTransition	Jedna transakce commitne; souběžné schválení → 409; právě 1 historie a 1 oznámení.	**PROŠEL · regresní test C07 na 31a22ad.**

BASELINE A CELKOVÉ OVĚŘENÍ

Testovaný zdroj: 31a22ad, založený na main@3c25e78; aktualizace důkazů 50a2909. Backend 37/37, frontend 10/10, typecheck/build prošly; lint 0 chyb / 1 varování. Smoke test: NOV0001 login, GET /api/slots = 12 a frontend HTTP 200. Fatima nezávisle ověřila start Compose, PostgreSQL 17.11 a 14 migrací do v14. Mocky C04 neprokazují commit ani souběh; to pokrývají samostatné PostgreSQL testy.

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0,
Time elapsed: 12.20 s -- in consultation_reservation_back
end.ReservationApprovalPersistenceTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0,
Time elapsed: 1.889 s -- in consultation_reservation_back
end.ReservationApprovalDesignTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0,
Time elapsed: 12.02 s -- in consultation_reservation_back
end.ReservationApprovalPersistenceTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

Nález C06 a technické rozhodnutí

Kandidátní nález F1: klient mohl podvrhnout identitu studenta při vytvoření rezervace.

NÁLEZ C06 F1 · PŘIJATO (ACCEPTED) · PŮVODNÍ BASELINE main@2b1e404

TECHNICKÁ REPRODUKCE · PŘED OPRAVOU

NOV0001	→	POST /api/reservations	→	HTTP 201 · PENDING
přihlášená role PEDAGOG		studentId = KRA0001		rezervace.student = KRA0001 historie.aktér = KRA0001

Očekávání: rezervace i auditní aktér odpovídají ověřené relaci. Pozorování na main@2b1e404: cizí student a nepravdivá historie.

OPRAVA A REGRESNÍ OVĚŘENÍ · SOUČASNÝ ZDROJ

Bearer session	→	ReservationController	→	ReservationService	→	Regression test
ověřená identita		předá AuthenticatedUser		student z session		vlastník + audit správně

DISPOSITION · PR / COMMIT · DŮKAZ OPRAVY

F1 PŘIJATO; PR #50 · fix commit 8ca6684 (předkem C07 31a22ad). studentId byl odstraněn z requestu; controller předává AuthenticatedUser a službě odvozuje studenta ze session. ReservationAuthorizationTests ověřuje, že podvržené ID nezmění vlastníka ani historie.aktér. Oprava uzavřena; F1 nemá otevřenou akci.